

UW PICO 5.09

File: path_waitpid.c

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>
#include <sys/stat.h>

#define MAX_ARGS 64
#define BUF_SIZE 512

// 1. PATH RESOLVER: Searches system PATH directories for executable binaries[cite: 18]
char* resolve_command_path(const char *cmd) {
    // If command already contains a path (e.g. ./program or /bin/ls)[cite: 18]
    if (strchr(cmd, '/')) {
        if (access(cmd, X_OK) == 0) {
            return strdup(cmd);
        }
        return NULL;
    }

    // Retrieve PATH environment variable[cite: 18]
    char *path_env = getenv("PATH");
    if (!path_env) return NULL;

    // Duplicate PATH string to avoid modifying original environment pointer[cite: 18]
    char *path_copy = strdup(path_env);
    char *dir = strtok(path_copy, ":");
    char full_path[BUF_SIZE];

    while (dir != NULL) {
        snprintf(full_path, sizeof(full_path), "%s/%s", dir, cmd);

        // Verify file existence and execution permissions (X_OK)[cite: 18]
        if (access(full_path, X_OK) == 0) {
            free(path_copy);
            return strdup(full_path); // Executable found[cite: 18]
        }
        dir = strtok(NULL, ":");
    }

    free(path_copy);
    return NULL; // Command missing or not executable[cite: 19]
}

// 2. PROCESS SYNCHRONIZATION: Launches child process and monitors with waitpid()[cite: 19]
void execute_and_monitor(char *args[]) {
    if (args[0] == NULL) return;

```

^G Get Help
 ^X Exit

^O WriteOut
 ^J Justify

^R Read File
 ^W Where is

^Y Prev Pg
 ^V Next Pg

^K Cut Text
 ^U UnCut Tex

UW PICO 5.09

File: path_waitpid.c

```

// Duplicate PATH string to avoid modifying original environment pointer[cite: 18]
char *path_copy = strdup(path_env);
char *dir = strtok(path_copy, ":");
char full_path[BUF_SIZE];

while (dir != NULL) {
    snprintf(full_path, sizeof(full_path), "%s/%s", dir, cmd);

    // Verify file existence and execution permissions (X_OK)[cite: 18]
    if (access(full_path, X_OK) == 0) {
        free(path_copy);
        return strdup(full_path); // Executable found[cite: 18]
    }
    dir = strtok(NULL, ":");
}

free(path_copy);
return NULL; // Command missing or not executable[cite: 19]
}

// 2. PROCESS SYNCHRONIZATION: Launches child process and monitors with waitpid()[cite: 19]
void execute_and_monitor(char *args[]) {
    if (args[0] == NULL) return;

    // Locate executable via PATH variable resolution[cite: 19]
    char *resolved_path = resolve_command_path(args[0]);
    if (!resolved_path) {
        printf("Error: Command '%s' not found or permission denied.\n", args[0]);
        return;
    }

    printf("[PATH Resolver] Resolved '%s' -> '%s'\n", args[0], resolved_path);

    pid_t pid = fork();
    if (pid < 0) {
        perror("Fork failed");
        free(resolved_path);
        return;
    } else if (pid == 0) {
        // Child Process: Execute resolved path[cite: 19]
        execv(resolved_path, args);
        perror("Execv failed");
        exit(EXIT_FAILURE);
    } else {
        // Parent Process: Monitor Child using waitpid()[cite: 19]
        int status;
        printf("[Parent] Monitoring Child Process PID: %d\n", pid);

```

^G Get Help
 ^X Exit

^O WriteOut
 ^J Justify

^R Read File
 ^W Where is

^Y Prev Pg
 ^V Next Pg

^K Cut Text
 ^U UnCut Tex

~ -- -zsh

~ -- -zsh

~ -- -zsh

~ -- path_waitpid

UW PICO 5.09File: path_waitpid.c

```
// Locate executable via PATH variable resolution[cite: 19]
char *resolved_path = resolve_command_path(args[0]);
if (!resolved_path) {
    printf("Error: Command '%s' not found or permission denied.\n", args[0]);
    return;
}

printf("[PATH Resolver] Resolved '%s' -> '%s'\n", args[0], resolved_path);

pid_t pid = fork();
if (pid < 0) {
    perror("Fork failed");
    free(resolved_path);
    return;
} else if (pid == 0) {
    // Child Process: Execute resolved path[cite: 19]
    execv(resolved_path, args);
    perror("Execv failed");
    exit(EXIT_FAILURE);
} else {
    // Parent Process: Monitor Child using waitpid()[cite: 19]
    int status;
    printf("[Parent] Monitoring Child Process PID: %d\n", pid);

    if (waitpid(pid, &status, 0) == -1) {
        perror("waitpid failed");
    } else {
        // Inspect Child Exit Status & Signals[cite: 20]
        if (WIFEXITED(status)) {
            printf("[Parent] Child PID %d exited normally with code: %d\n", pid, WEXITSTATUS(status));
        } else if (WIFSIGNALED(status)) {
            printf("[Parent] Child PID %d killed by signal: %d\n", pid, WTERMSIG(status));
        }
    }
}

free(resolved_path);
}

int main() {
    char input_buf[BUF_SIZE];

    while (1) {
        printf("\nPATH Shell > ");
        if (!fgets(input_buf, sizeof(input_buf), stdin)) break;

        input_buf[strcspn(input_buf, "\n")] = 0; // Remove newline[cite: 20]
        if (strcmp(input_buf, "exit") == 0) break;
    }
}
```

Get Help

Exit

WriteOut

Justify

Read File

Where is

Prev Pg

Next Pg

Cut Text

UnCut Tex

~ -- -zsh

~ -- -zsh

~ -- -zsh

~ -- path_waitpid

UW PICO 5.09File: path_waitpid.c

```
    if (waitpid(pid, &status, 0) == -1) {
        perror("waitpid failed");
    } else {
        // Inspect Child Exit Status & Signals[cite: 20]
        if (WIFEXITED(status)) {
            printf("[Parent] Child PID %d exited normally with code: %d\n", pid, WEXITSTATUS(status));
        } else if (WIFSIGNALED(status)) {
            printf("[Parent] Child PID %d killed by signal: %d\n", pid, WTERMSIG(status));
        }
    }
}

free(resolved_path);
}

int main() {
    char input_buf[BUF_SIZE];

    while (1) {
        printf("\nPATH Shell > ");
        if (!fgets(input_buf, sizeof(input_buf), stdin)) break;

        input_buf[strcspn(input_buf, "\n")] = 0; // Remove newline[cite: 20]
        if (strcmp(input_buf, "exit") == 0) break;

        // Simple space-delimited argument parser[cite: 20]
        char *args[MAX_ARGS];
        int arg_count = 0;
        char *token = strtok(input_buf, " ");

        while (token != NULL && arg_count < MAX_ARGS - 1) {
            args[arg_count++] = token;
            token = strtok(NULL, " ");
        }
        args[arg_count] = NULL;

        if (arg_count > 0) {
            execute_and_monitor(args);
        }
    }

    return 0;
}
```

Get Help

Exit

WriteOut

Justify

Read File

Where is

Prev Pg

Next Pg

Cut Text

UnCut Tex

~ — -zsh

~ — -zsh

~ — -zsh

~ — path_waitpid

```
medikondarakesh@Medikondas-MacBook-Air ~ % nano path_waitpid.c
medikondarakesh@Medikondas-MacBook-Air ~ % gcc -g -Wall path_waitpid.c -o path_waitpid
./path_waitpid
```

```
PATH Shell > /bin/date
[PATH Resolver] Resolved '/bin/date' -> '/bin/date'
[Parent] Monitoring Child Process PID: 2936
Fri Oct 2 08:45:08 IST 2026
[Parent] Child PID 2936 exited normally with code: 0
```

```
PATH Shell > nonexistentcommand123
Error: Command 'nonexistentcommand123' not found or permission denied.
```

```
PATH Shell > exit
medikondarakesh@Medikondas-MacBook-Air ~ % █
```